super_ambient

Preliminary architecture description (ArchiMate 3.1)

> **Status: preliminary, generated from the code, not reviewed.**
> Derived from commit `1409590` plus uncommitted changes on 2026-10-15. It describes what the repository contains, not what a deployment guarantees, and it is not a security or quality audit.

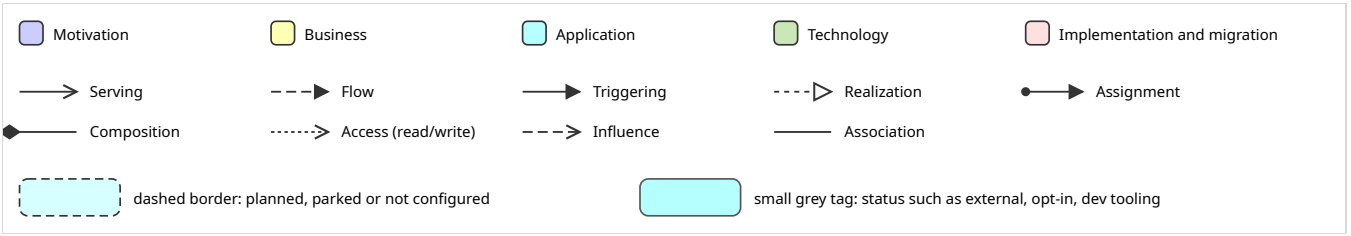
Model	98 elements, 161 relationships, 7 views (motivation 16, business 10, application 36, technology 22, implementation and migration 14)
Machine-readable model	`super_ambient-architecture.xml` (ArchiMate Open Group Exchange Format; opens in Archi via File, Import, Open Exchange XML Model)
Schema validation	passed (xmllint against the Open Group archimate3_Diagram.xsd, ArchiMate 3.1 exchange schema)
Archi import	NOT CHECKED. Archi was not available when this was generated. Relationship pairs were restricted to a conservative subset of the ArchiMate 3.1 relationship table.
Source of truth	`scripts/architecture/model.ts`; regenerate with `bun run architecture:build`

1 Purpose, scope and method

super_ambient is a local-first demo of a patient workspace on a public FHIR R4 test server. A clinician selects a *synthetic* patient and encounter, asks questions in Finnish and gets source-grounded answers, and turns a rehearsal conversation into a visit-note draft and structured proposals that are reviewed, approved and saved back as FHIR resources. Jev, a classification model from Typesafe, is used inside strict boundaries.

This document gives a first architecture view for discussion and for planning the move to an employer's Azure tenant. It was written from the repository: file layout, module boundaries, configuration defaults, tests and the decisions recorded in the project instructions. Every application component lists the files it is derived from, and the generator fails if a listed file does not exist.

- **Status tags** separate implemented parts from *opt-in*, *not configured*, *parked*, *planned* and *external* ones. Planned things are never drawn as implemented.
- **Notation.** The figures are simplified renderings with layer colours and a type tag; standard ArchiMate notation is available by opening the XML in a modelling tool. Legend:



Relationship counts: Composition 23, Assignment 9, Realization 17, Serving 15, Access 17, Influence 6, Triggering 7, Flow 28, Association 39.

2 Motivation

The driver is the documentation workload and patient data scattered across resources. The design is shaped less by features than by a handful of principles: nothing is saved without human approval, every answer and proposal is traceable to a source, the application and not a model builds FHIR, and secrets stay on the server. Two constraints bound the demo: only synthetic data on a public, unauthenticated server, and opt-in before transcript text goes to a third party.

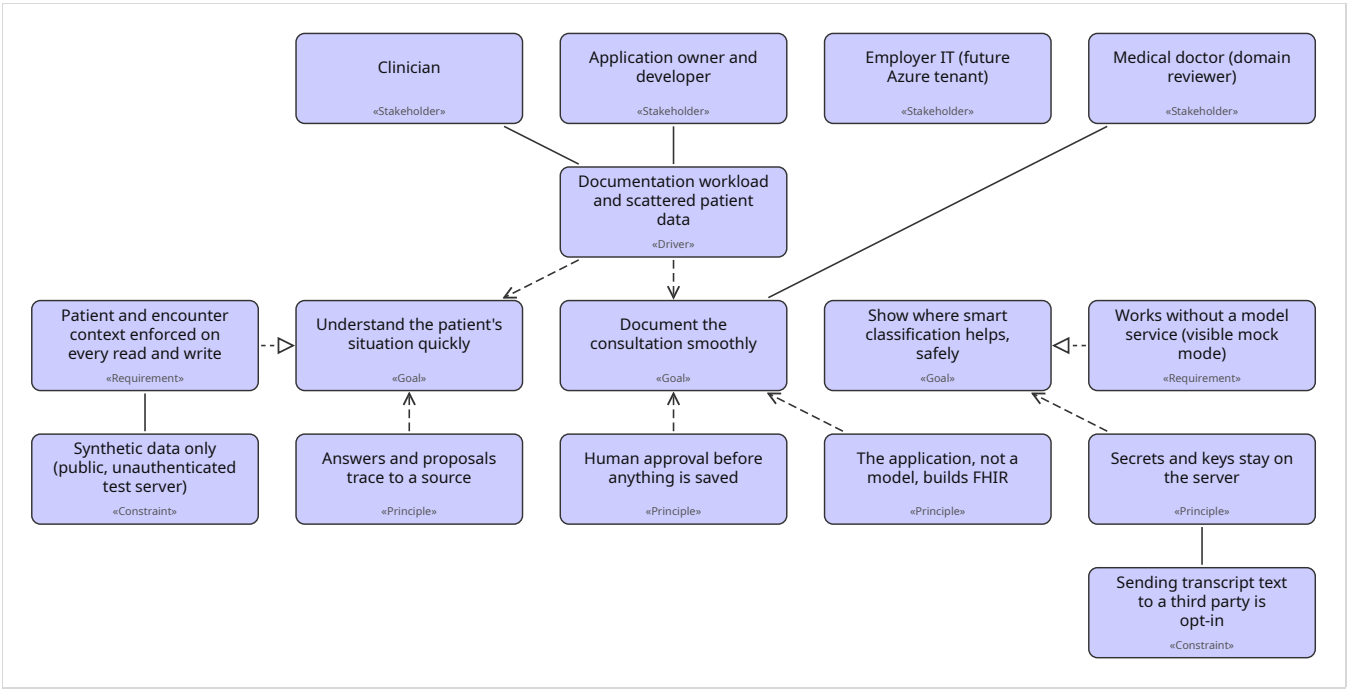


Figure 1. Motivation. Why the application exists: stakeholders, the driver, goals, and the principles, requirements and constraints that shape the design.

Element	Type	Status	Responsibility and source
Human approval before anything is saved	«Principle»	implemented	Proposals start unreviewed; edits revoke approval; the save path re-checks everything.
Answers and proposals trace to a source	«Principle»	implemented	Facts carry resource id and field; proposals carry an exact transcript quote.
The application, not a model, builds FHIR	«Principle»	implemented	Typed builders produce every saved resource; model output is never sent as a bundle.
Secrets and keys stay on the server	«Principle»	implemented	Keys are read from server-side environment variables only; errors never carry them.
Synthetic data only (public, unauthenticated test server)	«Constraint»	current	The public HAPI server has no privacy and is wiped periodically.
Sending transcript text to a third party is opt-in	«Constraint»	implemented	LLM_STRUCTURING=jev is off by default and warns in the UI.
Patient and encounter context enforced on every read and write	«Requirement»	implemented	Returned and followed resources are checked against the selected patient; writes only for demo-tagged patients. `lib/fhir/patient-guard.ts` `lib/workflow/save.ts`
Works without a model service (visible mock mode)	«Requirement»	implemented	Default provider is a rule-based mock that is labelled as such. `lib/clinical/language-adapter.ts`

3 Business layer

One role, four processes. *Ask about patient data* and *Record and document a consultation* are independent entry points; recording leads to *Review and approve proposals* and then *Save approved content to the EHR*. The application services and the recording view support the processes; the business objects are the visit note and the patient record.

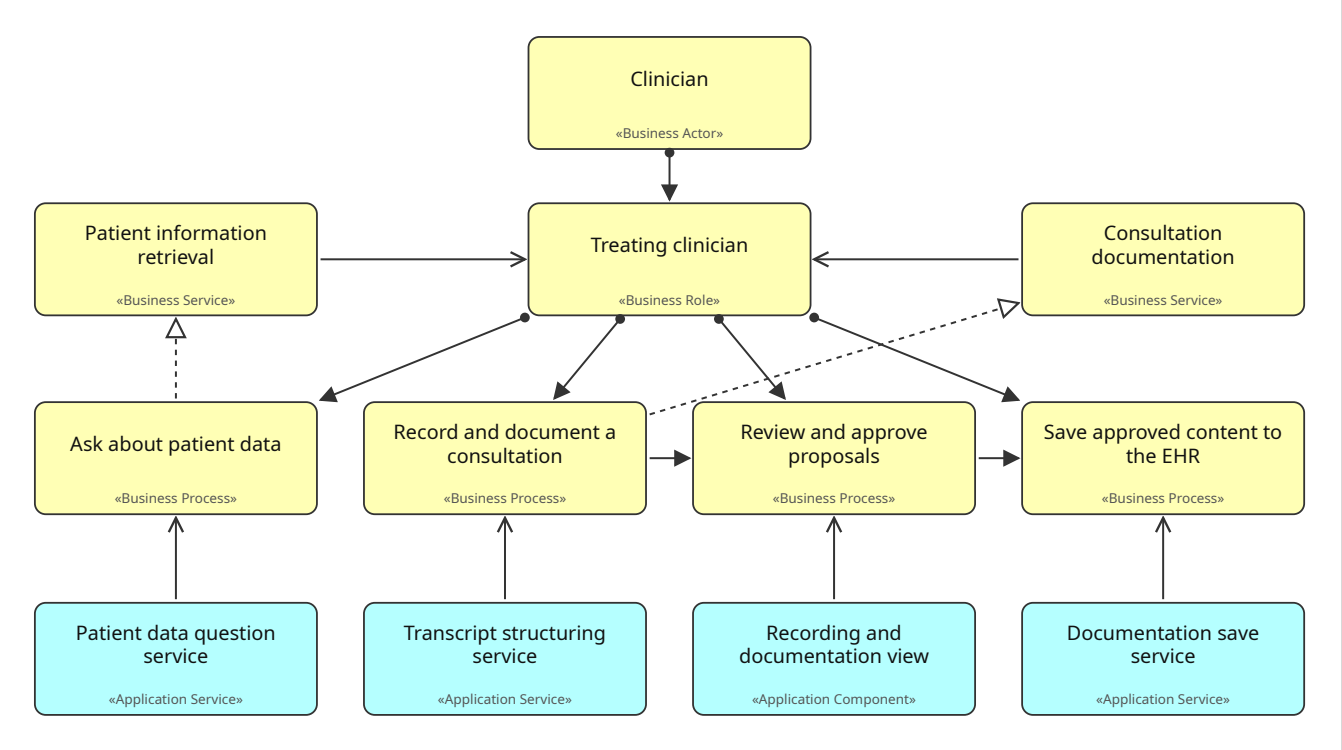


Figure 2. Business usage. How a clinician uses the application: the role, the four processes, the business services they realize, and the application services and views that support them.

4 Application layer

4.1 Structure

The application is a single Next.js project. The browser holds the React client; all FHIR access, model calls and secrets live in the server-side modules behind one internal REST interface. Business logic is separated from external connections by three adapter boundaries (EHR, language model, transcription), so the question-answering and structuring logic never touch HTTP or keys directly.

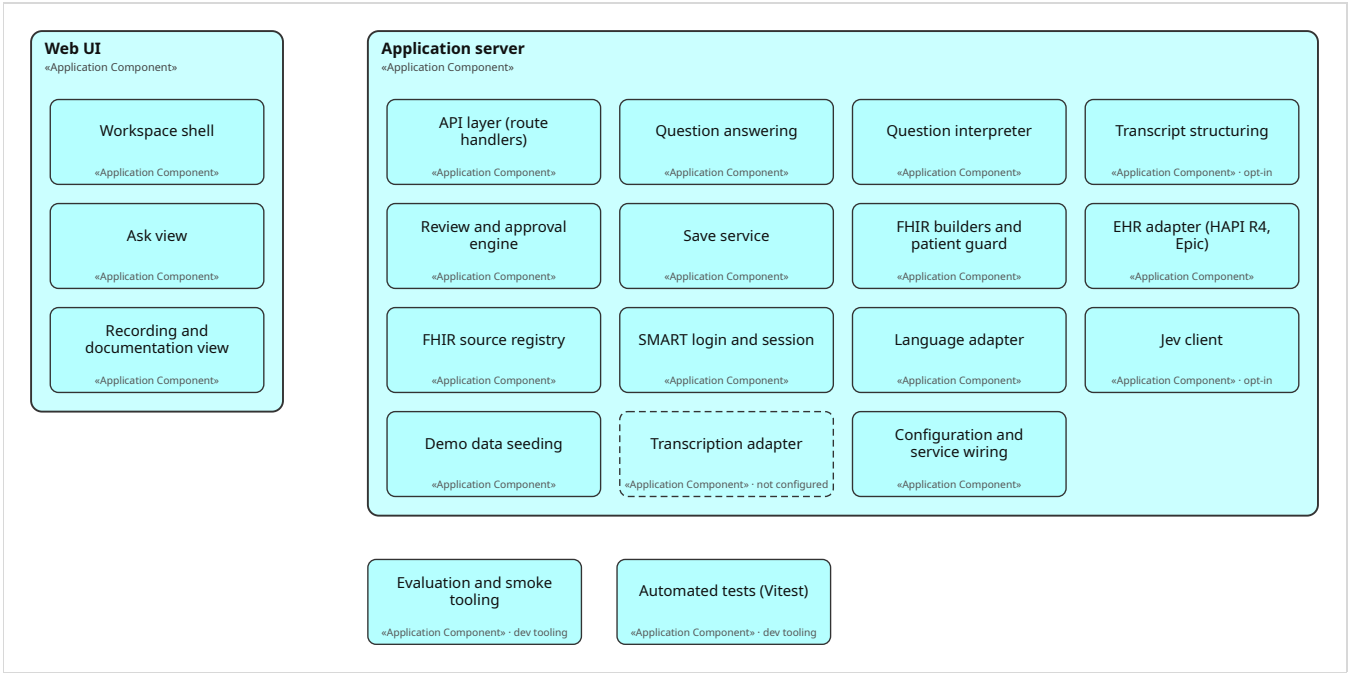


Figure 3. Application structure. Decomposition of the application: the web UI, the server-side modules with the source files they are built from, and the development tooling. Composition is shown by nesting.

4.2 Cooperation

A question flows from the UI through the API layer to question answering, which asks the language adapter to interpret it (Jev or the keyword mock), runs allow-listed read searches through the EHR adapter, extracts facts and composes the answer in code. Structuring takes a transcript, optionally classifies sentences through Jev, and hands proposals to the review engine. Saving runs only through the save service, which re-checks approvals, patient and encounter before one transaction.

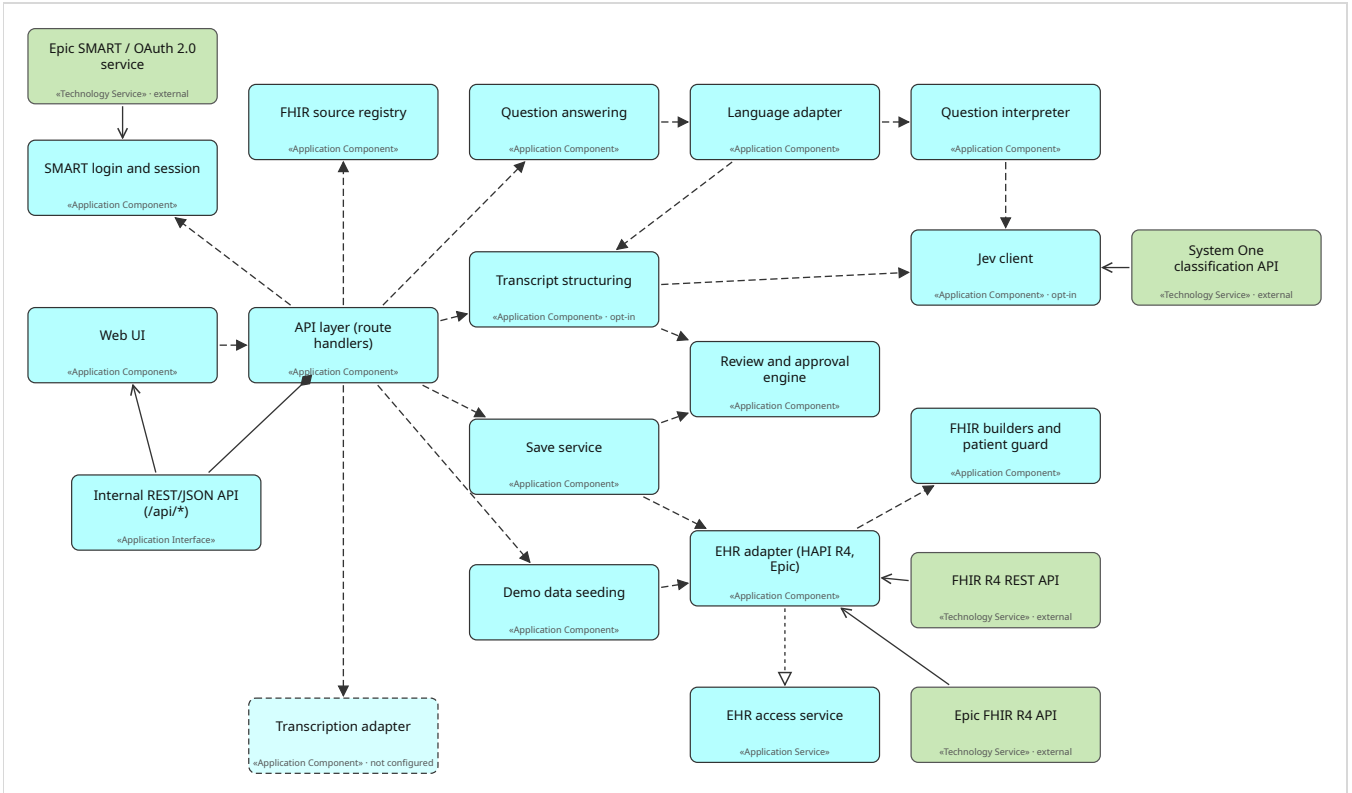


Figure 4. Application cooperation. Main information flows between the application components, and the external technology services they depend on (Jev, HAPI and Epic). Flows involving the configuration module are left out for readability; see the appendix.

4.3 Information

Information objects are short-lived application data (information need, facts, transcript, proposals, note draft, preview bundle). The only persistent data are the FHIR resources on the EHR server. A new Observation or Condition never carries a measurement time that was not stated, and the saved bundle carries per-resource identifiers derived from the previewed snapshot, so repeating a save creates no duplicates.

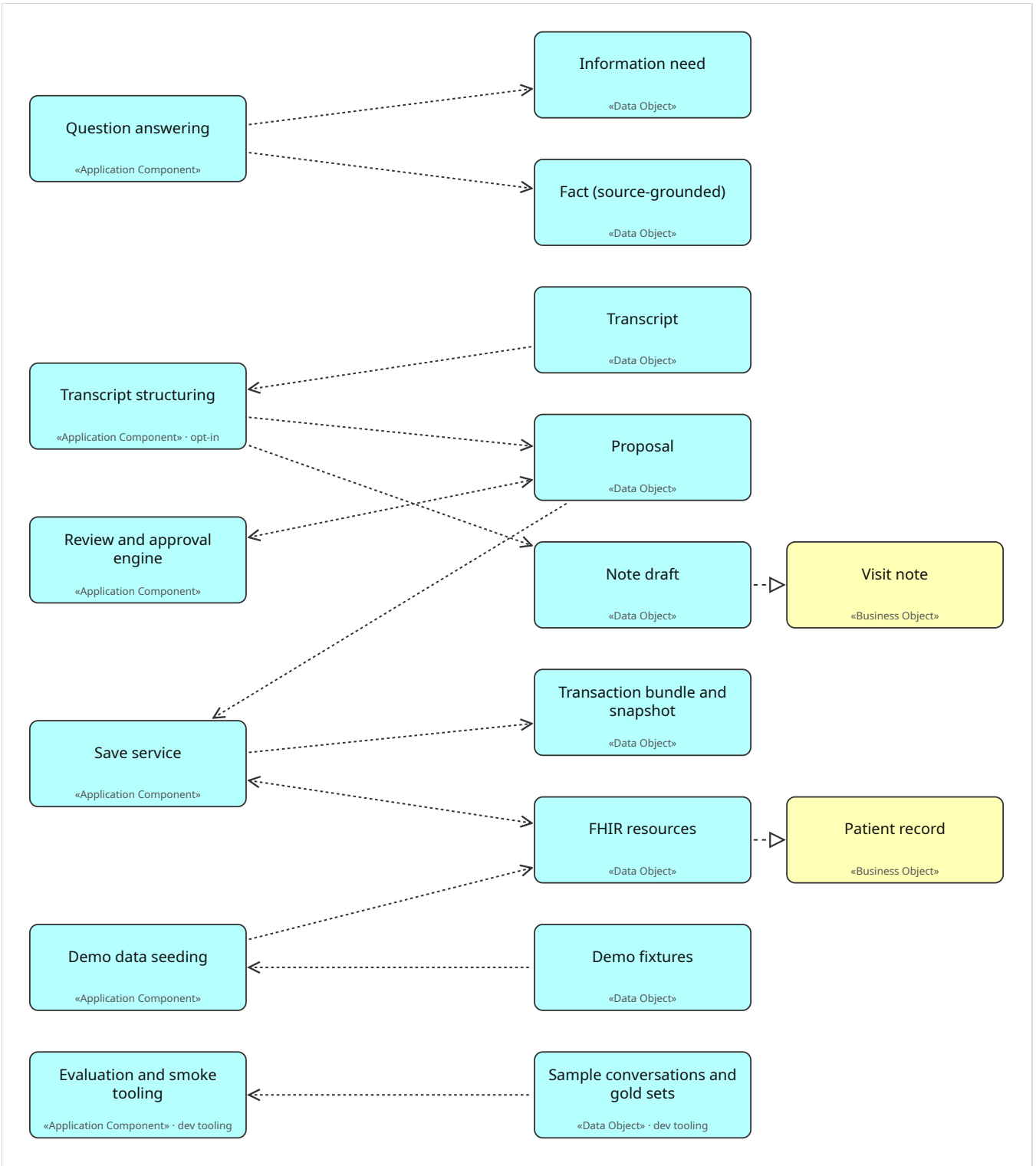


Figure 5. Information. Which components read and write which information objects, and how the data objects realize the business objects. Question answering also reads the FHIR resources (read access not drawn here; see the appendix). The FHIR resources are the only data persisted outside the application.

4.4 Components, interfaces and services

Element	Type	Status	Responsibility and source
Web UI	«Application Component»	implemented	React client: patient and visit selection, Ask view, recording and documentation view. `components/Workspace.tsx` `components/AskView.tsx` `components/RecordView.tsx` `components/ui.tsx` `components/api.ts`
Workspace shell	«Application Component»	implemented	Persistent context bar, patient search, encounter picker, status and seeding panels; guards against stale responses. `components/Workspace.tsx`
Ask view	«Application Component»	implemented	Question box, answer, source facts, search plan and technical details. `components/AskView.tsx`
Recording and documentation view	«Application Component»	implemented	Audio capture, transcript editor, sample conversations, proposal table, preview, save and read-back. `components/RecordView.tsx`
Application server	«Application Component»	implemented	Server-side logic in Next.js route handlers and the lib/ modules; holds all secrets and external connections. `app/api` `lib`
API layer (route handlers)	«Application Component»	implemented	Validates request bodies with strict schemas and delegates to the services. `app/api/ask/route.ts` `app/api/structure/route.ts` `app/api/save/route.ts` `app/api/save/preview/route.ts` `app/api/read/route.ts` `app/api/patients/route.ts` `app/api/encounters/route.ts` `app/api/status/route.ts`
Question answering	«Application Component»	implemented	Interprets a question, runs allow-listed read searches, extracts facts and composes a grounded answer. `lib/clinical/ask.ts` `lib/clinical/facts.ts` `lib/clinical/intents.ts`
Question interpreter	«Application Component»	implemented	Maps a question to an information need: Jev classification or keyword mock. `lib/clinical/jev-interpreter.ts` `lib/clinical/mock-language.ts`
Transcript structuring	«Application Component»	opt-in	Turns a transcript into a note draft and proposals. Mock rules by default; Jev classifies sentences when LLM_STRUCTURING=jev. `lib/clinical/jev-structurer.ts` `lib/clinical/sentence-classifier.ts` `lib/clinical/mock-structurer.ts` `lib/clinical/sentences.ts` `lib/clinical/measurements.ts` `lib/clinical/text-rules.ts`
Review and approval engine	«Application Component»	implemented	Approval hashes, staleness, allowed FHIR targets and validation of structuring output. `lib/workflow/review.ts`
Save service	«Application Component»	implemented	Server-side re-check, preview, validation, one transaction with conditional create, read-back and timeout recovery. `lib/workflow/save.ts`
FHIR builders and patient guard	«Application Component»	implemented	Allow-listed search builders, typed resource builders and the patient-scope guard. `lib/fhir/search-builders.ts` `lib/fhir/resource-builders.ts` `lib/fhir/patient-guard.ts`

Element	Type	Status	Responsibility and source
EHR adapter (HAPI R4, Epic)	«Application Component»	implemented	EhrAdapter interface and its R4 implementation: paging, limits, retries, capability check, optional Bearer token. One shared instance for sources without login; a new instance per request for sources with login. `lib/fhir/ehr-adapter.ts` `lib/fhir/hapi-adapter.ts` `lib/fhir/http.ts` `lib/services.ts`
FHIR source registry	«Application Component»	implemented	The only place that defines a source: address, login type, write permission and patient-search form. The browser sends only a source id; unknown ids are rejected and read-only sources refuse writes. `lib/fhir/sources.ts`
SMART login and session	«Application Component»	implemented	Authorization code flow with PKCE (S256) as a public client; endpoints discovered from the server and restricted to its origin. The access token lives in an httpOnly cookie that only the server reads; no server-side session store. `lib/auth/pkce.ts` `lib/auth/smart.ts` `lib/auth/session.ts` `lib/auth/errors.ts` `app/api/auth/[source]/login/route.ts` `app/api/auth/[source]/callback/route.ts` `app/api/auth/[source]/logout/route.ts`
Language adapter	«Application Component»	implemented	Selects mock, Jev hybrid or unavailable and reports per capability who does what. `lib/clinical/language-adapter.ts`
Jev client	«Application Component»	opt-in	Timeout, bounded retries, strict response parsing; errors never carry key or text. `lib/clinical/jev-client.ts` `lib/jev-settings.ts`
Demo data seeding	«Application Component»	implemented	User-triggered upload of the app's own synthetic fixtures (second write path next to the save service). `app/api/seed/route.ts` `lib/fhir/fixtures.ts`
Transcription adapter	«Application Component»	not configured	Interface for speech-to-text; no provider is wired, so transcripts are pasted by hand. `lib/transcription/adapter.ts` `app/api/transcribe/route.ts`
Configuration and service wiring	«Application Component»	implemented	Reads environment settings and creates the adapters. `lib/config.ts` `lib/services.ts`
Evaluation and smoke tooling	«Application Component»	dev tooling	Eval scripts (mock and live Jev), HAPI smoke test and browser walkthrough. `scripts/eval-interpret.ts` `scripts/eval-structure.ts` `scripts/smoke-hapi.ts` `scripts/ui-e2e.ts`
Automated tests (Vitest)	«Application Component»	dev tooling	Unit and integration tests against an in-memory FHIR fake and a scripted Jev; no UI or route tests. `tests`
Internal REST/JSON API (/api/*)	«Application Interface»	implemented	The only interface between browser and server. `app/api`
Patient data question service	«Application Service»	implemented	POST /api/ask.
Transcript structuring service	«Application Service»	implemented	POST /api/structure.
Documentation save service	«Application Service»	implemented	POST /api/save/preview, /api/save and /api/read.
EHR access service	«Application Service»	implemented	Patient, encounter and resource reads and the write transaction.

4.5 Data objects

Element	Type	Status	Responsibility and source
Information need	«Data Object»	implemented	Strict schema: intent, resource types, time range, scope. Never contains a URL or patient.
Fact (source-grounded)	«Data Object»	implemented	A value or statement with resource id, fields and clinical time.
Transcript	«Data Object»	implemented	Speaker-labelled segments with a content version.
Proposal	«Data Object»	implemented	Reviewable suggestion: type, value, exact source quote, certainty, review status.
Note draft	«Data Object»	implemented	Reason, history, findings, assessment and plan sections.
Transaction bundle and snapshot	«Data Object»	implemented	The previewed bundle and the hash the server re-checks.
FHIR resources	«Data Object»	implemented	Patient, Encounter, Observation, Condition, DocumentReference and read-only types.
Demo fixtures	«Data Object»	implemented	Two synthetic patients with visits, vitals, labs, medication, allergy, problem and a report. `lib/fhir/fixtures.ts`
Sample conversations and gold sets	«Data Object»	dev tooling	14 synthetic conversations and question sets; written by an AI, not reviewed by a doctor. `lib/clinical/sample-conversations.ts` `scripts/data/question-eval-set.ts`

4.6 Integration points and trust boundaries

Connection	Protocol	Data	Notes
Browser to application server	HTTPS, JSON, /api/*	Questions, transcripts, proposals, save requests	Boundary 1. No authentication in the demo; protect the deployment (e.g. Vercel Deployment Protection).
Server to HAPI FHIR R4	HTTPS REST, application/fhir+json	Searches, reads, $validate, one write transaction	Boundary 2. Public, shared server: synthetic data only. Writes only for demo-tagged patients.
Browser to Epic login	HTTPS redirect (authorize), then redirect back to /api/auth/epic/callback	OAuth code flow with PKCE; the user types credentials on Epic's page only	Boundary 2b. The app never sees the credentials. The registered redirect URI must match exactly.
Server to Epic on FHIR	HTTPS REST (Bearer token); token request to the OAuth endpoint	Reads for the selected patient; code exchange with the PKCE verifier	Boundary 2c. Read-only; synthetic sandbox data. Token kept in an httpOnly cookie, about one hour; no refresh.
Server to Typesafe Jev	HTTPS POST /v1/systemone, Bearer key	Question text; with LLM_STRUCTURING=jev also transcript sentences and two previous turns	Boundary 3. Third-party processing; terms not yet checked. Responses are typed answers only.
Browser to microphone	MediaRecorder, local	Rehearsal audio, kept in browser memory	Never sent to HAPI; sent to a transcription service only on an explicit action (none is configured).
Server to ICD-API (parked)	HTTP, local service	Not wired in this repository	Would be local-only; unreachable from Vercel.
Dev tooling to HAPI / Typesafe	smoke:hapi, e2e:ui (writes), eval --jev (sends text)	Synthetic data	Permission-gated; the owner runs the live evaluations.

5 Technology and deployment

The same build runs on a developer workstation (Bun), on Vercel today, and is intended for an Azure tenant later. The ICD-API service is a local systemd unit outside this repository and is unreachable from Vercel, which is why ICD-11 suggestions are parked. Configuration is environment-driven; defaults keep every external service off.

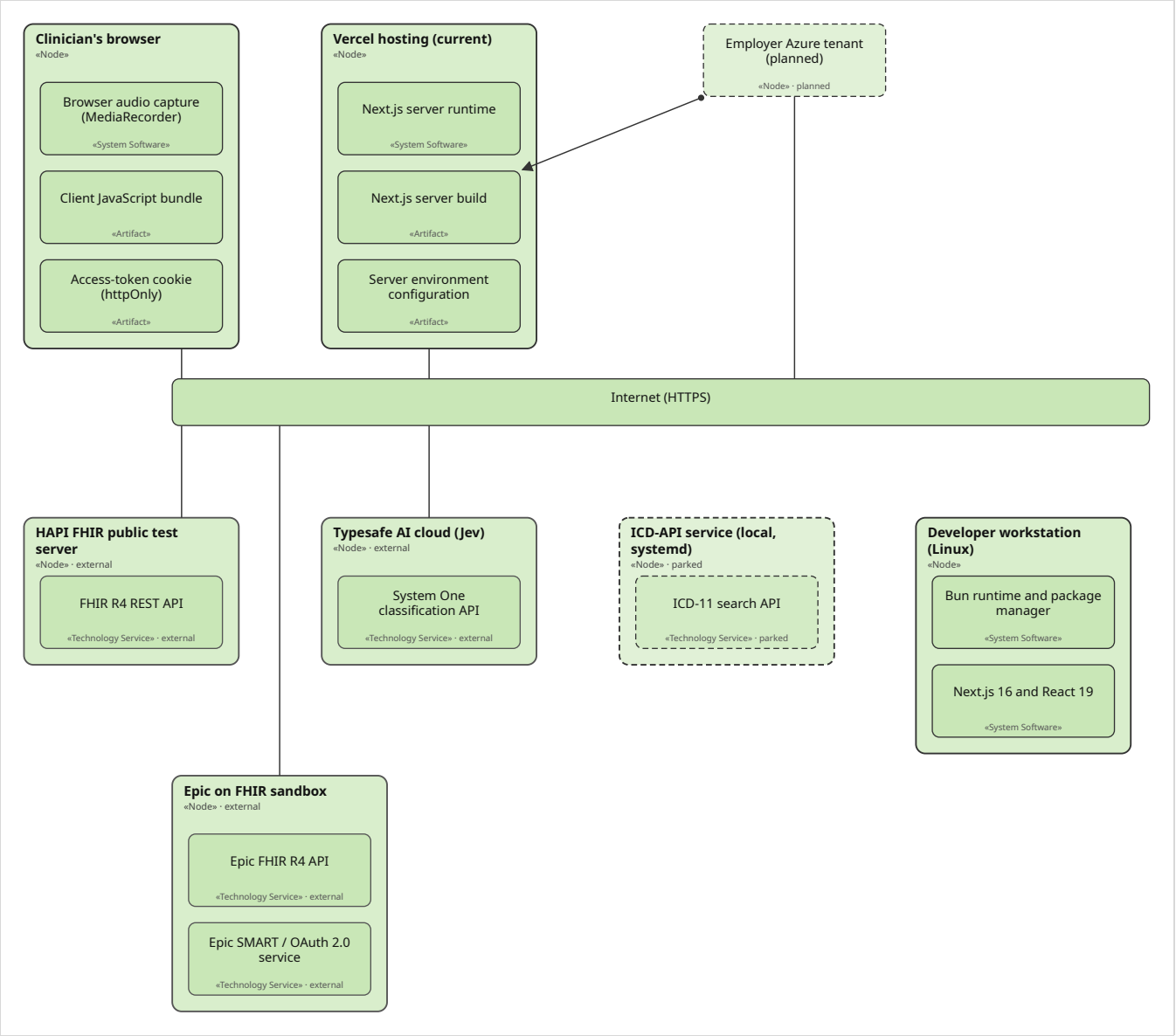


Figure 6. Technology and deployment. Where the application runs and what it talks to. Planned and parked elements are drawn dashed. Node-to-node flows (browser to Vercel to HAPI and Typesafe) are in the model but not drawn here.

5.1 Configuration

Setting	Default	Meaning
`FHIR_BASE_URL`	https://hapi.fhir.org/baseR4	FHIR server; https assumed; paging links are checked against this origin and path.
`FHIR_TIMEOUT_MS`	20000	Per request timeout; GET retried twice, POST never retried automatically.
`EPIC_CLIENT_ID`	unset	Epic sandbox client id (not secret, kept in the environment). Without it the Epic source shows a message and login is unavailable.
`LLM_PROVIDER`	mock	mock, jev or none. Jev without a key is reported as unavailable, never guessed.

Setting	Default	Meaning
`LLM_STRUCTURING`	mock	jev sends transcript sentences (no identifiers) to Typesafe. Only effective when LLM_PROVIDER=jev.
`TYPESAFE_API_KEY (LLM_API_KEY fallback)`	unset	Server-side secret; never exposed to the browser or in errors.
`LLM_ENDPOINT / LLM_MODEL`	https://api.typesafe.ai/v1/ systemone / jev-latest	Endpoint must be https.
`JEV_CONCURRENCY`	1	1 to 8 parallel Jev requests; 1 suits the entry plan.
`TRANSCRIPTION_PROVIDER`	none	No provider is implemented; the transcript is pasted by hand.

5.2 Technology elements

Element	Type	Status	Responsibility
Clinician's browser	«Node»	current	Runs the React client; captures audio locally and keeps it in memory.
Browser audio capture (MediaRecorder)	«System Software»	implemented	Audio is never sent to HAPI; it goes to a transcription service only on an explicit action.
Client JavaScript bundle	«Artifact»	implemented	Output of next build.
Access-token cookie (httpOnly)	«Artifact»	implemented	Holds the OAuth access token for a source that needs login (about one hour). SameSite=Strict, Secure on https; unreadable by page scripts.
Developer workstation (Linux)	«Node»	current	bun run dev; no Node installed, Bun runs everything.
Bun runtime and package manager	«System Software»	current	bun@1.4.2, bun.lock.
Next.js 16 and React 19	«System Software»	current	App Router with cacheComponents.
Vercel hosting (current)	«Node»	current	Hosts the deployed demo; not protected by login unless Deployment Protection is on.
Next.js server runtime	«System Software»	current	Serverless functions; capability cache is per instance.
Next.js server build	«Artifact»	implemented	Route handlers and server modules.
Server environment configuration	«Artifact»	implemented	Host environment variables or a local env file; holds the Jev key and the Epic client id.
Employer Azure tenant (planned)	«Node»	planned	Target hosting; may also run an ICD-API VM.
HAPI FHIR public test server	«Node»	external	https://hapi.fhir.org/baseR4; shared, unauthenticated, wiped periodically.
FHIR R4 REST API	«Technology Service»	external	Search, read, $validate and transaction.
Epic on FHIR sandbox	«Node»	external	https://fhir.epic.com/interconnect-fhir-oauth; synthetic test patients, login with documented test users; the app reads only.
Epic FHIR R4 API	«Technology Service»	external	Patient (demographics search needs a birth date with the name), Encounter, Observation and other reads.
Epic SMART / OAuth 2.0 service	«Technology Service»	external	authorize and token endpoints from .well-known/smart-configuration; PKCE S256, standalone launch, public client.
Typesafe AI cloud (Jev)	«Node»	external	https://api.typesafe.ai/v1/systemone; data-processing terms not yet checked.
System One classification API	«Technology Service»	external	Typed questions (choice, score, noul) over a state; no text generation.
ICD-API service (local, systemd)	«Node»	parked	Runs on the owner's machine (port 8382); unreachable from Vercel.
ICD-11 search API	«Technology Service»	parked	Possible future code suggestions; ICD-10 would be needed for production.
Internet (HTTPS)	«Communication Network»	current	All browser-to-server and server-to-service traffic.

6 Cross-cutting concerns

6.1 Quality attributes

Attribute	How it is addressed today

Attribute	How it is addressed today
Safety	Human approval; edits revoke approval; server re-checks snapshot, patient and encounter; conservative targets (no FHIR target unless the model's judgement is decisive).
Traceability	Facts carry resource id and field; proposals carry an exact transcript quote and speaker; saved resources keep a proposal-derived identifier.
Privacy and security	Server-side keys, strict request schemas, plain-text rendering, no automatic attachment fetching, minimal logging. Epic login uses PKCE, the token stays in an httpOnly cookie and is never logged or put in errors. The application itself has no login or audit trail yet.
Adaptability	Three adapter boundaries (EHR, language, transcription) and a source registry; Epic was added as a second source without a new code path, but another EHR may still need its own search variants and compatibility tests.
Testability	207 deterministic tests against an in-memory FHIR fake and a scripted Jev; separate live evaluations. No UI or route-handler tests.
Performance	Jev calls about 275 to 310 ms each (measured in evaluations); structuring is one call per sentence or clause. No load testing.
Availability	Failures are shown, not hidden; the mock fallback is an explicit per-question choice. Capability cache is per server instance.

6.2 Key decisions

#	Decision	Rationale
D1	Adapter boundaries (EhrAdapter, ClinicalLanguageAdapter, TranscriptionAdapter)	Swap EHR, model or speech service without touching UI or workflow logic.
D2	Treat Jev as a classifier, not a generator	Jev returns typed decisions only, so the app keeps extraction, quotes and values in code and uses Jev for judgement. Hallucinated values become impossible by construction.
D3	Human approval and staleness before any write	Clinical content must be reviewed; any edit or context change invalidates earlier approvals.
D4	Mock mode by default and visible labelling	The demo works without keys; mock is never presented as a language model.
D5	FHIR built only by the application	Typed builders and strict schemas; model output never becomes a bundle.
D6	Opt-in for sending transcript text to a third party	Question text and transcript sentences have different privacy weight.
D7	Public HAPI with self-created synthetic fixtures	Reproducible demo data; conditional create prevents duplicates; seeding is user-triggered.
D8	ICD-11 suggestions parked	Nice-to-have for the demo; production would need ICD-10 and Finnish coverage that ICD-11 MMS lacks.
D9	Vercel now, employer Azure tenant later	Nothing in the code is host-specific; endpoints, keys and timeouts are configuration.
D10	Source registry; the browser sends only a source id	Addresses, login type and write permission cannot be chosen by the client. A new source is a registry entry; a read-only source refuses writes on the server.
D11	Access token in an httpOnly cookie, no server-side session	Works on serverless hosts without shared memory or a session secret, and keeps the token away from page scripts. Cost: about an hour of validity, no refresh, a new adapter instance per request.
D12	Epic is read-only	The sandbox is not ours to write to, and the demo-tag rule that protects HAPI does not apply there. A registry test requires that writable sources need no login.

7 Roadmap, risks and open questions

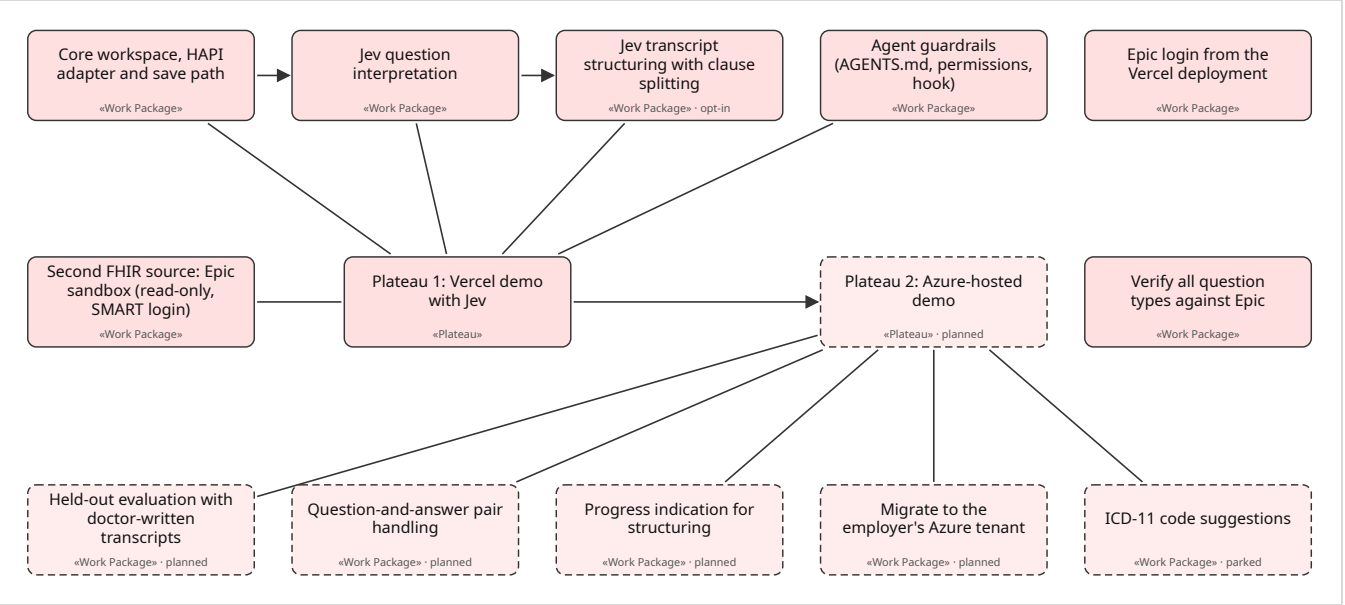


Figure 7. Implementation and migration. What has been delivered, and what is planned or parked on the way from the Vercel demo to an Azure-hosted demo. Nothing here is committed scope.

Element	Type	Status	Responsibility and source
Core workspace, HAPI adapter and save path	«Work Package»	implemented	Delivered first version of the specification.
Jev question interpretation	«Work Package»	implemented	Classifier-based interpretation with thresholds and an evaluation set.
Jev transcript structuring with clause splitting	«Work Package»	opt-in	Opt-in hybrid structuring; evaluation on synthetic gold set.
Agent guardrails (AGENTS.md, permissions, hook)	«Work Package»	implemented	Project instructions, permission rules and an .env hook. `AGENTS.md` `.claude/settings.json` `.claude/hooks/block-env-reads.sh`
Second FHIR source: Epic sandbox (read-only, SMART login)	«Work Package»	implemented	Source registry and selector, PKCE login with cookie session, Epic search shapes and test-patient list. Verified by the owner for login, patient search, encounters and questions on blood pressure, labs, allergies, medications, conditions, imaging reports, documents and the overview (medication names are read from the reference display; an imaging document is recognised from its type; Epic needs a category on Observation searches, so the overview splits observations into vitals and labs). `lib/fhir/sources.ts` `lib/auth`
Verify all question types against Epic	«Work Package»	implemented	Every question type has been tried by hand. Findings: Epic ignores _sort, requires a category or code on Observation searches and offers no MedicationStatement in R4 (403); the app reports the 403 on purpose and explains 401 and 403 clearly.
Epic login from the Vercel deployment	«Work Package»	implemented	Callback URI registered at Epic, client id set in Vercel, code deployed. The owner confirmed that login works from the deployed app. Epic took about ten minutes to accept the new URI, and it compares the URI literally (https, exact path).
Held-out evaluation with doctor-written transcripts	«Work Package»	planned	The real accuracy check; the current sets are in-sample.

Element	Type	Status	Responsibility and source
Question-and-answer pair handling	«Work Package»	planned	Short replies such as 'Ei.' need the preceding question.
Progress indication for structuring	«Work Package»	planned	About 300 ms per sentence at concurrency 1.
Migrate to the employer's Azure tenant	«Work Package»	planned	Hosting, secrets store, identity and network rules.
ICD-11 code suggestions	«Work Package»	parked	Nice-to-have; needs a Finnish-to-English vocabulary step.
Plateau 1: Vercel demo with Jev	«Plateau»	current	Today's state.
Plateau 2: Azure-hosted demo	«Plateau»	planned	Target state, still a demo.

7.1 Risks and technical debt

#	Observation	Suggested response
R1	UI components and route handlers have no unit tests; the browser walkthrough covers one desktop size in mock mode; the Jev UI path was verified manually once.	Add route tests and a Jev-mode walkthrough with a scripted server.
R2	Evaluation sets and thresholds were written and tuned by an AI on the same data; reported accuracy is optimistic.	Held-out, doctor-written transcripts, run once without tuning.
R3	The demo has no authentication. A public URL could spend the Jev quota and write demo data.	Deployment protection now; proper identity on Azure.
R4	Two write paths (clinical save and demo seeding) plus permission-gated scripts.	Keep the documented invariant; add tests around the seed route.
R5	Third-party processing at Typesafe is unreviewed; transcript sentences leave the system when opted in.	Check terms and region before any non-synthetic use.
R6	Structuring latency about 300 ms per unit at concurrency 1; hosting time limits may bite.	Progress indication, higher concurrency, streaming.
R7	Short replies ('Ei.') are not interpreted; clause splitting only at mutta, vaikka and semicolons.	Question-and-answer pair handling.
R8	The public HAPI is shared and wiped; non-demo patients can be read (read-only).	Re-seed on demand; consider a private FHIR server.
R9	No CI; checks run manually.	Propose CI as a separate change.
R10	No audit trail or structured logging; one console.error only.	Required before any real use.
R11	Per-instance caches on serverless hosts give inconsistent status and skipped parameter checks until the first capability check. Epic adapters are built per request and never run the parameter check.	Check capabilities on first use per instance or cache externally.
R12	Epic reads are verified by hand for a few sandbox patients: login, patient search, encounters and questions on blood pressure, labs, allergies, medications, conditions, imaging reports, documents and the overview work. Nothing can be tested automatically because the login needs a browser.	Re-test by hand after changes to searches; add Epic-specific search variants where Epic rejects ours.
R13	Epic access tokens last about an hour and are not refreshed. Every new hostname (preview deployments, a custom domain, the Azure tenant) needs its callback URI registered at Epic first, and Epic compares it literally.	Register each origin before use; consider a refresh token only if the one-hour limit gets in the way.
R14	Epic's patient search finds by name only together with a birth date; the exact accepted combinations are not documented in what was read.	Test more combinations; prefer identifier lookup or launch context for real use.

#	Observation	Suggested response
R15	Epic ignores _sort, so a capped search (60 resources, 3 pages) is not guaranteed to contain the newest records; the app sorts what it gets and reports partial searches. Epic R4 also offers no MedicationStatement (403), which is shown in medication answers on purpose.	Page further or filter by date for Epic where it matters; add the MedicationStatement API if Epic ever offers it for R4.

7.2 Open questions

1. Should adjacent separate systolic and diastolic Observations ever be shown or stored as one reading? (A doctor will decide.)
2. Which code system is needed first in a real setting (ICD-10 is expected), and who maintains the Finnish term mapping?
3. What does the employer's Azure tenant require: region, data-processing terms for Typesafe, identity (Entra ID?), network rules and secrets storage?
4. Which real EHR and FHIR profile are the target, and what authentication and write permissions apply? Epic's sandbox is read-only here; would clinician apps launch from inside the EHR (EHR launch) instead of a standalone login?
5. Retention and handling of audio and transcripts, and who may see Jev responses.
6. Which concurrency does the Typesafe plan allow, and what is the acceptable wait for structuring?

Appendix A: Motivation and business elements

Element	Type	Status	Responsibility
Clinician	«Stakeholder»	current	Reads patient data and documents consultations; the intended end user.
Application owner and developer	«Stakeholder»	current	Decides scope, approves plans and runs the live evaluations.
Medical doctor (domain reviewer)	«Stakeholder»	current	Not yet involved in review. Will advise on blood pressure pairing and clinical labels.
Employer IT (future Azure tenant)	«Stakeholder»	current	Will host the application in a later phase.
Documentation workload and scattered patient data	«Driver»	current	Clinicians spend time finding data and writing notes.
Understand the patient's situation quickly	«Goal»	current	Ask questions in plain language and get source-grounded answers.
Document the consultation smoothly	«Goal»	current	Turn a conversation into a reviewable note and structured suggestions.
Show where smart classification helps, safely	«Goal»	current	Use Jev as a classifier inside strict, human-reviewed boundaries.
Clinician	«Business Actor»	current	The person using the workspace.
Treating clinician	«Business Role»	current	Role that asks, records, reviews and saves.
Ask about patient data	«Business Process»	implemented	Pose a question, read the answer with its source observations.
Record and document a consultation	«Business Process»	implemented	Record or paste a rehearsal conversation and build a note draft.
Review and approve proposals	«Business Process»	implemented	Edit, approve or reject each structured suggestion and the note.
Save approved content to the EHR	«Business Process»	implemented	Preview, save as one transaction, and read the resources back.
Patient information retrieval	«Business Service»	implemented	Answers to questions about the selected patient.
Consultation documentation	«Business Service»	implemented	Draft note and structured proposals from a conversation.
Visit note	«Business Object»	implemented	The consultation note (käyntimerkintä); saved as a DocumentReference with docStatus preliminary.
Patient record	«Business Object»	implemented	The patient's data held in the EHR (the public HAPI server in the demo).

Appendix B: All relationships

Complete list, including the relationships that are not drawn in a figure. Composition between the Web UI, the application server and their parts is shown by nesting in Figure 3.

Source	Relationship	Target
Clinician	Association	Documentation workload and scattered patient data
Application owner and developer	Association	Documentation workload and scattered patient data
Medical doctor (domain reviewer)	Association	Document the consultation smoothly
Employer IT (future Azure tenant)	Association	Migrate to the employer's Azure tenant
Documentation workload and scattered patient data	Influence	Understand the patient's situation quickly
Documentation workload and scattered patient data	Influence	Document the consultation smoothly
Answers and proposals trace to a source	Influence	Understand the patient's situation quickly
Human approval before anything is saved	Influence	Document the consultation smoothly
The application, not a model, builds FHIR	Influence	Document the consultation smoothly
Secrets and keys stay on the server	Influence	Show where smart classification helps, safely
Patient and encounter context enforced on every read and write	Realization	Understand the patient's situation quickly
Works without a model service (visible mock mode)	Realization	Show where smart classification helps, safely
Synthetic data only (public, unauthenticated test server)	Association	Patient and encounter context enforced on every read and write
Sending transcript text to a third party is opt-in	Association	Secrets and keys stay on the server
Clinician	Assignment	Treating clinician
Treating clinician	Assignment	Ask about patient data
Treating clinician	Assignment	Record and document a consultation
Treating clinician	Assignment	Review and approve proposals
Treating clinician	Assignment	Save approved content to the EHR
Record and document a consultation	Triggering	Review and approve proposals
Review and approve proposals	Triggering	Save approved content to the EHR
Ask about patient data	Realization	Patient information retrieval
Record and document a consultation	Realization	Consultation documentation
Patient information retrieval	Serving	Treating clinician
Consultation documentation	Serving	Treating clinician
Ask about patient data	Access (Read)	Patient record
Record and document a consultation	Access (Write)	Visit note
Review and approve proposals	Access (ReadWrite)	Visit note
Save approved content to the EHR	Access (Write)	Patient record
Note draft	Realization	Visit note
FHIR resources	Realization	Patient record
Patient data question service	Serving	Ask about patient data
Transcript structuring service	Serving	Record and document a consultation
Documentation save service	Serving	Save approved content to the EHR

Source	Relationship	Target
Recording and documentation view	Serving	Review and approve proposals
Ask view	Serving	Ask about patient data
Web UI	Composition	Workspace shell
Web UI	Composition	Ask view
Web UI	Composition	Recording and documentation view
Application server	Composition	API layer (route handlers)
Application server	Composition	Question answering
Application server	Composition	Question interpreter
Application server	Composition	Transcript structuring
Application server	Composition	Review and approval engine
Application server	Composition	Save service
Application server	Composition	FHIR builders and patient guard
Application server	Composition	EHR adapter (HAPI R4, Epic)
Application server	Composition	FHIR source registry
Application server	Composition	SMART login and session
Application server	Composition	Language adapter
Application server	Composition	Jev client
Application server	Composition	Demo data seeding
Application server	Composition	Transcription adapter
Application server	Composition	Configuration and service wiring
API layer (route handlers)	Flow	FHIR source registry
API layer (route handlers)	Flow	SMART login and session
SMART login and session	Flow	EHR adapter (HAPI R4, Epic)
FHIR source registry	Flow	EHR adapter (HAPI R4, Epic)
Configuration and service wiring	Association	FHIR source registry
Configuration and service wiring	Association	SMART login and session
SMART login and session	Association	Access-token cookie (httpOnly)
API layer (route handlers)	Composition	Internal REST/JSON API (/api/*)
Internal REST/JSON API (/api/*)	Serving	Web UI
Web UI	Flow	API layer (route handlers)
API layer (route handlers)	Flow	Question answering
API layer (route handlers)	Flow	Transcript structuring
API layer (route handlers)	Flow	Save service
API layer (route handlers)	Flow	Demo data seeding
API layer (route handlers)	Flow	Transcription adapter
API layer (route handlers)	Flow	EHR adapter (HAPI R4, Epic)
Question answering	Flow	Language adapter
Language adapter	Flow	Question interpreter
Language adapter	Flow	Transcript structuring
Question interpreter	Flow	Jev client
Transcript structuring	Flow	Jev client
Question answering	Flow	EHR adapter (HAPI R4, Epic)
Question answering	Flow	FHIR builders and patient guard

Source	Relationship	Target
Transcript structuring	Flow	Review and approval engine
Save service	Flow	Review and approval engine
Save service	Flow	EHR adapter (HAPI R4, Epic)
Save service	Flow	FHIR builders and patient guard
Demo data seeding	Flow	EHR adapter (HAPI R4, Epic)
EHR adapter (HAPI R4, Epic)	Flow	FHIR builders and patient guard
Configuration and service wiring	Association	Language adapter
Configuration and service wiring	Association	EHR adapter (HAPI R4, Epic)
Configuration and service wiring	Association	Jev client
Question answering	Realization	Patient data question service
Transcript structuring	Realization	Transcript structuring service
Save service	Realization	Documentation save service
EHR adapter (HAPI R4, Epic)	Realization	EHR access service
EHR access service	Serving	Question answering
EHR access service	Serving	Save service
EHR access service	Serving	Demo data seeding
Question answering	Access (Write)	Information need
Question answering	Access (Write)	Fact (source-grounded)
Question answering	Access (Read)	FHIR resources
Transcript structuring	Access (Read)	Transcript
Transcript structuring	Access (Write)	Proposal
Transcript structuring	Access (Write)	Note draft
Review and approval engine	Access (ReadWrite)	Proposal
Save service	Access (Read)	Proposal
Save service	Access (Write)	Transaction bundle and snapshot
Save service	Access (ReadWrite)	FHIR resources
Demo data seeding	Access (Read)	Demo fixtures
Demo data seeding	Access (Write)	FHIR resources
Evaluation and smoke tooling	Access (Read)	Sample conversations and gold sets
Recording and documentation view	Association	Sample conversations and gold sets
Clinician's browser	Composition	Browser audio capture (MediaRecorder)
Developer workstation (Linux)	Composition	Bun runtime and package manager
Developer workstation (Linux)	Composition	Next.js 16 and React 19
Vercel hosting (current)	Composition	Next.js server runtime
Clinician's browser	Assignment	Client JavaScript bundle
Vercel hosting (current)	Assignment	Next.js server build
Employer Azure tenant (planned)	Assignment	Next.js server build
Client JavaScript bundle	Realization	Web UI
Next.js server build	Realization	Application server
Server environment configuration	Association	Configuration and service wiring
Server environment configuration	Association	Next.js server build

Source	Relationship	Target
HAPI FHIR public test server	Realization	FHIR R4 REST API
Typesafe AI cloud (Jev)	Realization	System One classification API
ICD-API service (local, systemd)	Realization	ICD-11 search API
FHIR R4 REST API	Serving	EHR adapter (HAPI R4, Epic)
System One classification API	Serving	Jev client
Epic on FHIR sandbox	Realization	Epic FHIR R4 API
Epic on FHIR sandbox	Realization	Epic SMART / OAuth 2.0 service
Epic FHIR R4 API	Serving	EHR adapter (HAPI R4, Epic)
Epic SMART / OAuth 2.0 service	Serving	SMART login and session
Clinician's browser	Assignment	Access-token cookie (httpOnly)
Vercel hosting (current)	Flow	Epic on FHIR sandbox
Internet (HTTPS)	Association	Epic on FHIR sandbox
Clinician's browser	Flow	Vercel hosting (current)
Vercel hosting (current)	Flow	HAPI FHIR public test server
Vercel hosting (current)	Flow	Typesafe AI cloud (Jev)
Internet (HTTPS)	Association	Clinician's browser
Internet (HTTPS)	Association	Vercel hosting (current)
Internet (HTTPS)	Association	Employer Azure tenant (planned)
Internet (HTTPS)	Association	HAPI FHIR public test server
Internet (HTTPS)	Association	Typesafe AI cloud (Jev)
ICD-API service (local, systemd)	Association	ICD-11 code suggestions
Core workspace, HAPI adapter and save path	Triggering	Jev question interpretation
Jev question interpretation	Triggering	Jev transcript structuring with clause splitting
Core workspace, HAPI adapter and save path	Association	Plateau 1: Vercel demo with Jev
Jev question interpretation	Association	Plateau 1: Vercel demo with Jev
Jev transcript structuring with clause splitting	Association	Plateau 1: Vercel demo with Jev
Agent guardrails (AGENTS.md, permissions, hook)	Association	Plateau 1: Vercel demo with Jev
Held-out evaluation with doctor-written transcripts	Association	Plateau 2: Azure-hosted demo
Question-and-answer pair handling	Association	Plateau 2: Azure-hosted demo
Progress indication for structuring	Association	Plateau 2: Azure-hosted demo
Migrate to the employer's Azure tenant	Association	Plateau 2: Azure-hosted demo
ICD-11 code suggestions	Association	Plateau 2: Azure-hosted demo
Plateau 1: Vercel demo with Jev	Triggering	Plateau 2: Azure-hosted demo
Second FHIR source: Epic sandbox (read-only, SMART login)	Association	Plateau 1: Vercel demo with Jev
Verify all question types against Epic	Association	Plateau 1: Vercel demo with Jev
Epic login from the Vercel deployment	Association	Plateau 1: Vercel demo with Jev
Second FHIR source: Epic sandbox (read-only, SMART login)	Association	SMART login and session
Second FHIR source: Epic sandbox (read-only, SMART login)	Association	FHIR source registry
Second FHIR source: Epic sandbox (read-only, SMART login)	Triggering	Verify all question types against Epic

Source	Relationship	Target
Second FHIR source: Epic sandbox (read-only, SMART login)	Triggering	Epic login from the Vercel deployment
Migrate to the employer's Azure tenant	Association	Employer Azure tenant (planned)
Jev question interpretation	Association	Question interpreter
Jev transcript structuring with clause splitting	Association	Transcript structuring

Appendix C: Using and regenerating the model

- **Open in Archi:** File, Import, Open Exchange XML Model, then choose `super_ambient-architecture.xml`. Other tools that read the Open Group exchange format can import the same file. The views come with positions; elements carry `Status` and `Source` properties.
- **Regenerate:** edit `scripts/architecture/model.ts` and run `bun run architecture:build`. Validation fails on unknown elements, relationship pairs outside the allowed subset, missing source files, undrawable edges and elements that appear in no view.
- **Schema check (optional):** set `ARCHIMATE_XSD_DIR` to a folder with the Open Group ArchiMate 3.1 XSD files and `xml.xsd` to validate with `xmllint` during generation.
- **Known limits:** relationship validity was checked against a conservative subset, not the complete ArchiMate relationship table; the file has not been opened in a modelling tool; the figures are simplified renderings.